

Programación
Orientado a Objetos

PROYECTO VIDEOJUEGO

POKEMON



Julian Alexis
Sánchez Sánchez

Alan Cortez
Hernandez

Javler Rojas
Vivanco

Irvin Perez Ochoa

Contenido

.....	1
Introducción	3
Objetivo	3
Diagramas UML	4
Diagrama 1. Diagrama general de clases (estructural)	4
Diagrama UML 1.1 Estructura Principal (Pokemon y Batalla)	5
Diagrama UML 2. Herencia de Elementos y Ataques	6
Diagrama UML 3. Caso de Uso del Sistema Pokémon	7
Diagrama UML 4. Diagrama de modelo de equipos	8
Diagrama UML 5. Diagrama de Clases del Bloque “Ataque/Daño”	9
Diagrama UML 6. Diagrama caso de uso “Batalla de Pokemones”	10
Diagrama UML 7. Caso de Uso: Proceso de Ataque	11
Diagrama UML 8. Caso de Uso: Ciclo de Batalla	12
Diagrama UML 9. Caso de Uso: Gestión de Equipos	13
Diagrama UML 10. Diagrama de clases: Pokémon vs Habilidades	14
Diagrama UML 11. Diagrama de Caso de Uso: “Simulador complete”	15
DOCUMENTACIÓN DEL CÓDIGO	16
Detalles del código	16
Código de JavaScript:	17
Explicación del Código	24
Ejecución del programa:	34
Conslusión	36
Agradecimiento	36

Introducción

En este proyecto desarrollamos un simulador de combate por turnos inspirado en el sistema de batalla de Pokémon, aplicando principios de Programación Orientada a Objetos. Definimos clases para representar Pokémon, elementos, ataques y la mecánica completa de la batalla. Nuestro objetivo fue crear una simulación funcional y lógica que refleje interacciones reales entre objetos y su comportamiento dinámico durante un combate. Este trabajo representa nuestra comprensión de los conceptos fundamentales de POO como herencia, composición, encapsulamiento y polimorfismo.

Objetivo

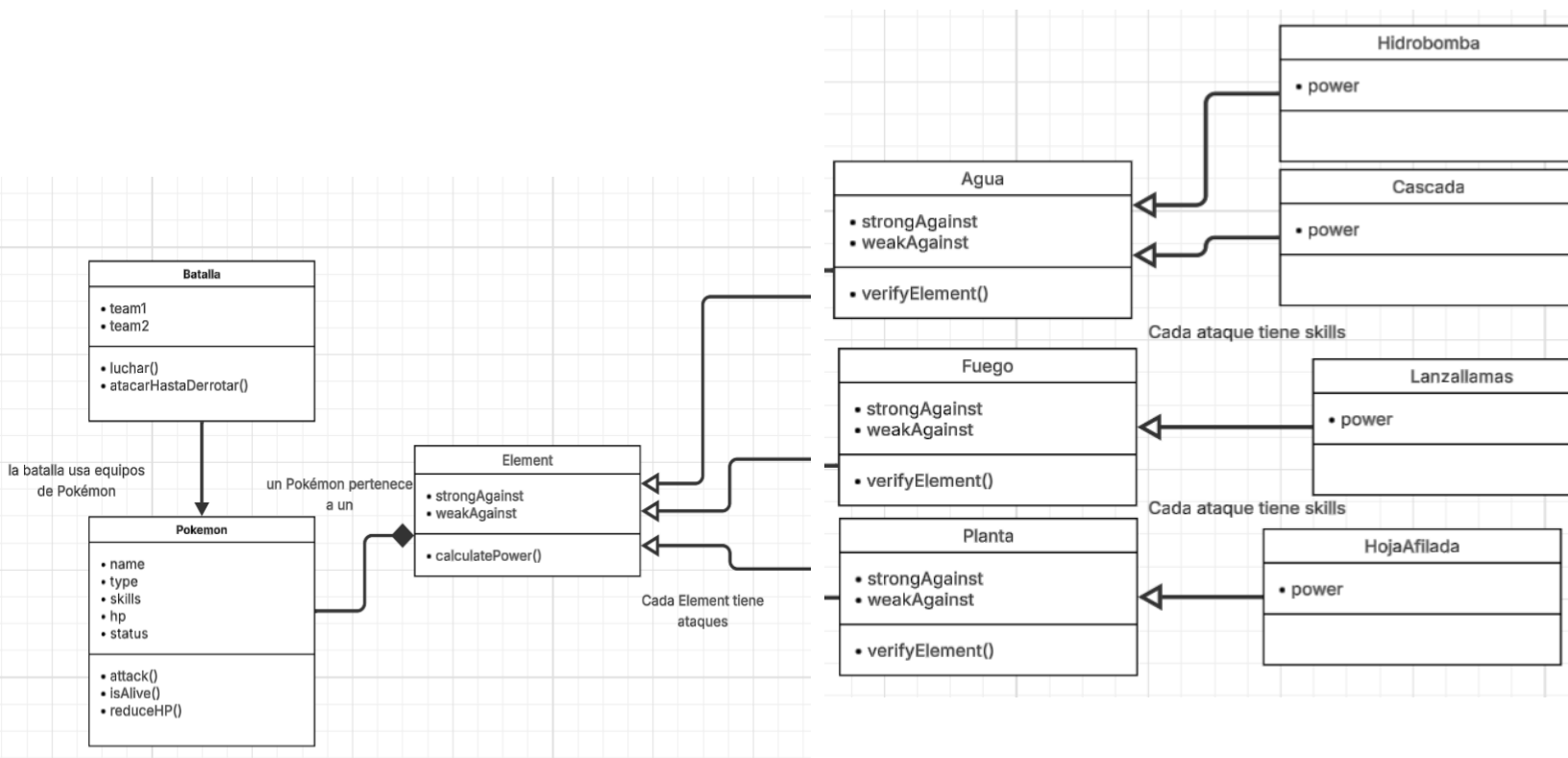
Realizar una simulación de un sistema de combate por turnos en el que dos equipos de Pokémon se enfrenten, aplicando los conceptos de herencia de clases, composición de objetos y cálculo dinámico de daño según la relación entre elementos.

El duelo será por equipos como en el tradicional juego de Pokémon.

Diagramas UML

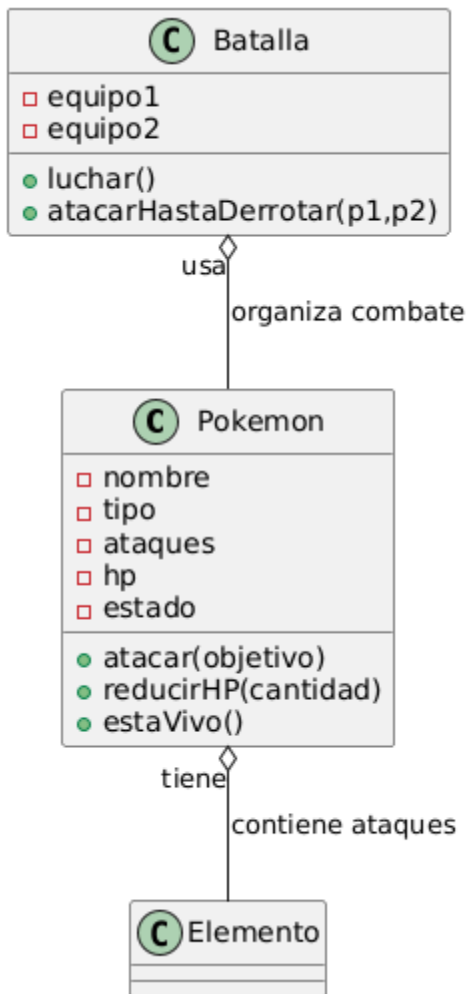
Aquí los diagramas UML que realizamos para la visualización y diseño estructural funcional del sistema desarrollado. Estos diagramas permiten visualizar la arquitectura del código y las relaciones entre las clases, cómo se comunican entre sí y qué responsabilidades tiene cada una dentro del modelo. A través de ellos se observa la aplicación de conceptos clave de POO como herencia, composición y encapsulamiento. Estos diagramas sirven como representación gráfica previa al desarrollo de la lógica en programación.

Diagrama 1. Diagrama general de clases (estructural)



Este diagrama de clases muestra las clases principales que se usaran y como se relacionan o heredan los atributos y métodos de la clase padre. Aquí batalla depende de la clase Pokemon, ya que para una batalla se usan equipos de Pokemon, mientras que cada pokemon pertenece a un elemento, mientras que cada elemento tiene ataques únicos, que heredan sus atributos, estos ataques tienen skills únicos, en el que heredan los atributos y métodos del tipo de ataque de cada uno.

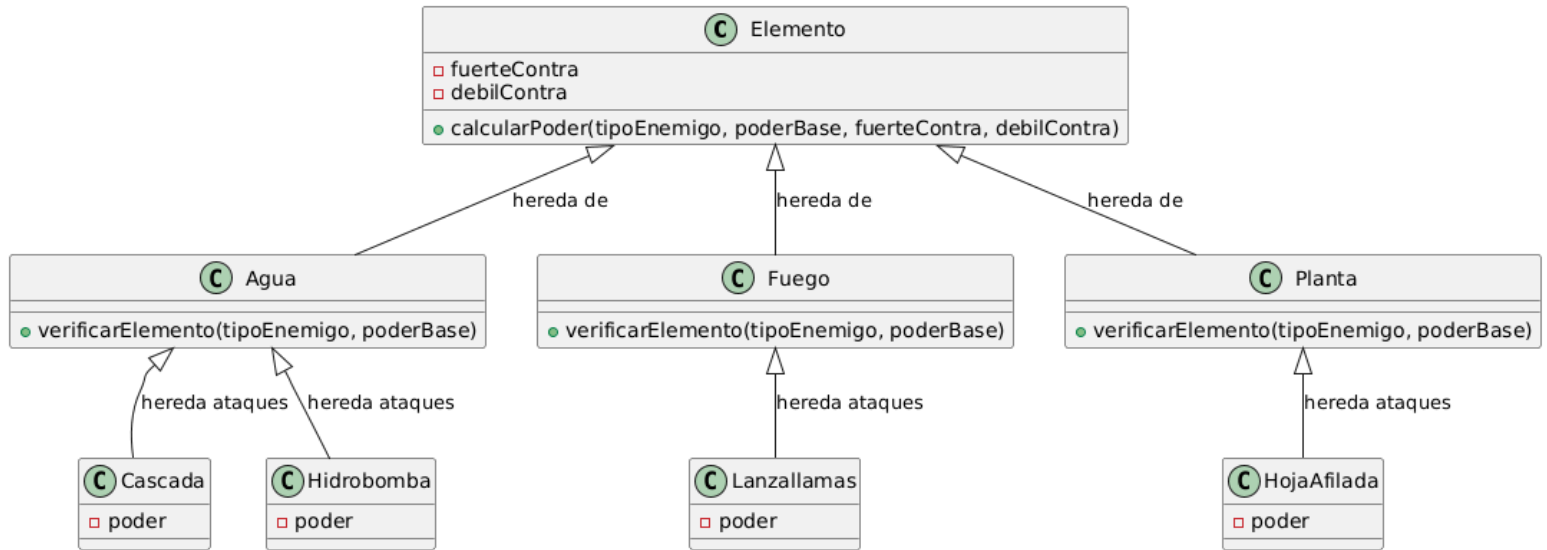
Diagrama UML 1.1 Estructura Principal (Pokemon y Batalla)



En este diagrama vemos cómo se relacionan las clases principales del proyecto. La clase Pokémon define las características básicas de un Pokémon y contiene una lista de ataques. La clase Batalla, por su parte, administra los equipos y utiliza los objetos Pokémon para ejecutar la lucha. Aquí se observa una relación de composición porque la batalla no puede existir sin los Pokémon que participan en ella. Este modelo refleja el principio de modularidad y abstracción en la programación orientada a objetos.

Aquí se muestra la composición del sistema:
un Pokémon contiene ataques y la clase Batalla usa Pokémon para ejecutar la simulación de combate

Diagrama UML 2. Herencia de Elementos y Ataques

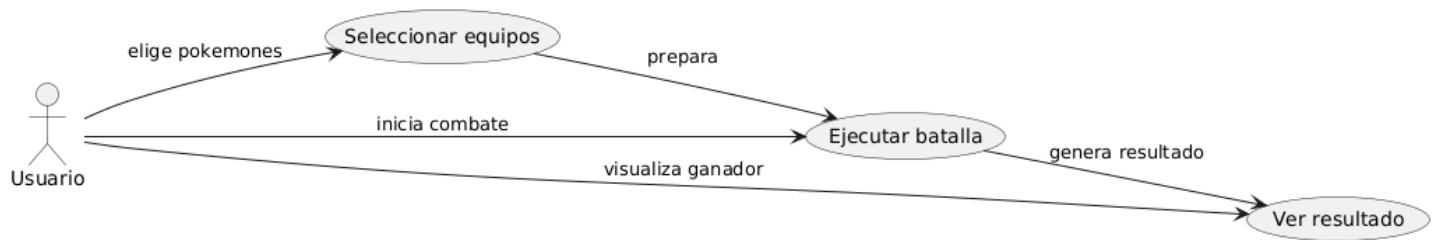


En este diagrama podemos observar la jerarquía de herencia en el sistema de elementos. La clase **Elemento** funciona como la clase padre que define el comportamiento base para calcular el poder de un ataque. Las clases **Agua**, **Fuego** y **Planta** heredan de ella e implementan su lógica particular de ventajas y debilidades. Finalmente, los ataques concretos como **Cascada**, **Hidrobomba** y **Lanzallamas** heredan de sus respectivos tipos, lo que permite una estructura flexible y reutilizable gracias a la programación orientada a objetos.

Este diagrama muestra la estructura jerárquica del sistema de elementos y ataques.

Las clases hijas especializan el comportamiento de la clase padre, aplicando polimorfismo.

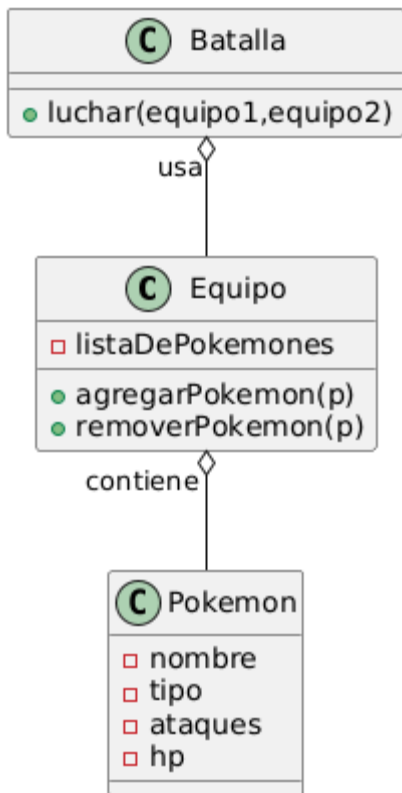
Diagrama UML 3. Caso de Uso del Sistema Pokémon



En este diagrama se presenta la interacción del usuario con el sistema. El usuario selecciona los equipos, inicia la batalla y visualiza el resultado final. Este tipo de diagrama no representa estructura interna, sino acciones que el usuario puede realizar. Nos ayuda a entender el sistema desde el punto de vista de la funcionalidad y la experiencia del usuario.

Este diagrama presenta el uso del programa desde el punto de vista del usuario. Representa cómo el usuario interactúa con el sistema para generar una batalla y ver resultados.

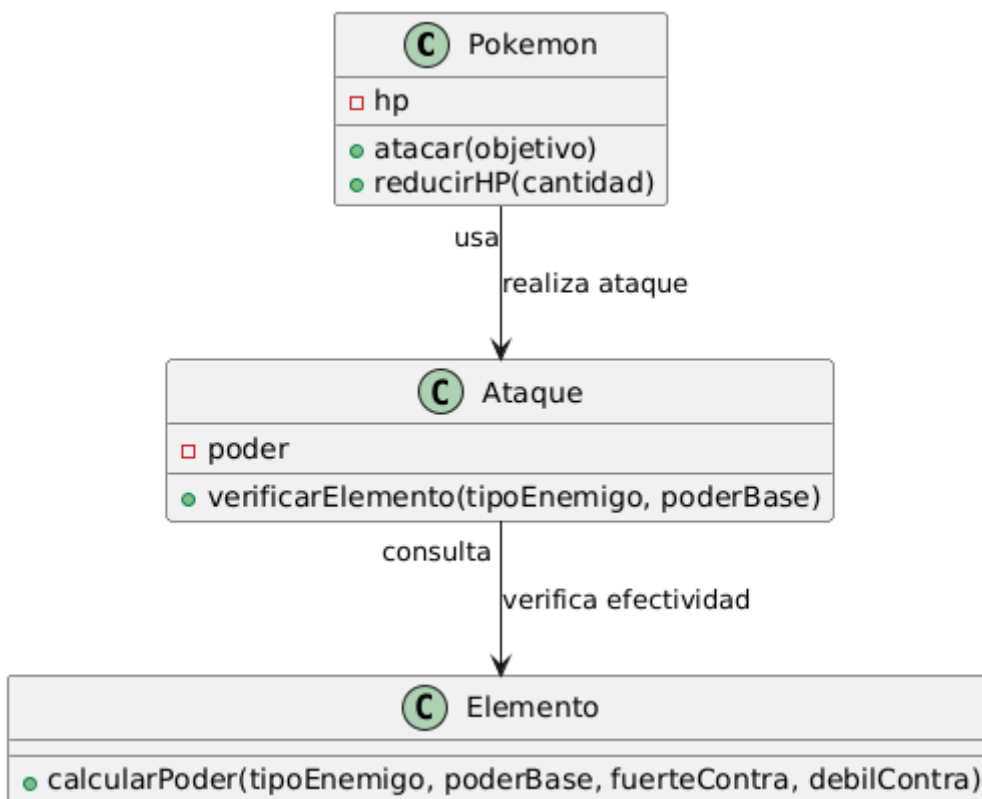
Diagrama UML 4. Diagrama de modelo de equipos



En este diagrama se abstrae el concepto de equipo como una clase, aunque en el código no existe explícitamente. Sin embargo, esta representación permite entender cómo se agrupan los Pokémon dentro de cada equipo y cómo la clase Batalla utiliza esos equipos para llevar a cabo la lucha. Este diagrama es útil para explicar el modelo mental del programa y justificar la arquitectura.

Este diagrama expresa la organización de equipos como una clase conceptual. En lugar de instancias concretas, se generaliza y se muestra la estructura lógica del sistema.

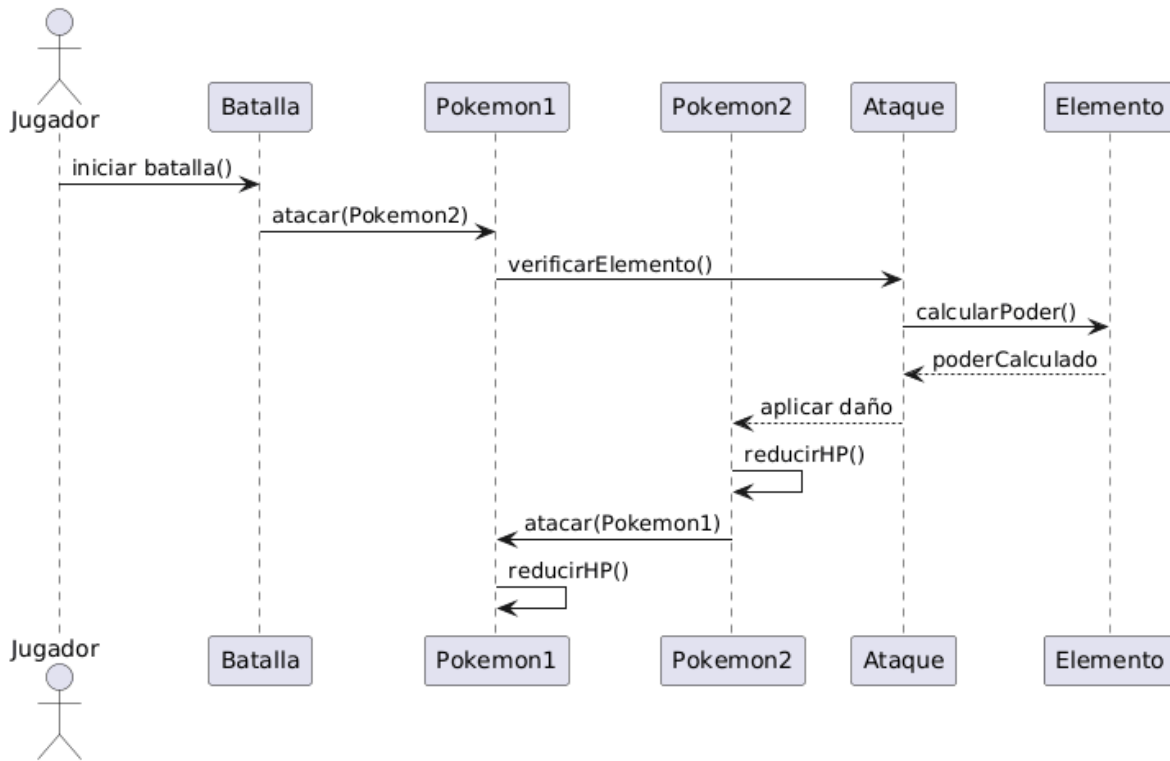
Diagrama UML 5. Diagrama de Clases del Bloque “Ataque/Daño”



Este diagrama muestra la interacción entre un Pokémon, su ataque y el elemento del ataque. Cuando el Pokémon ejecuta un ataque, se consulta al elemento para verificar efectividad y calcular el daño real. Este diseño aprovecha el polimorfismo al permitir que la verificación de elementos se realice de manera diferente según el tipo del ataque.

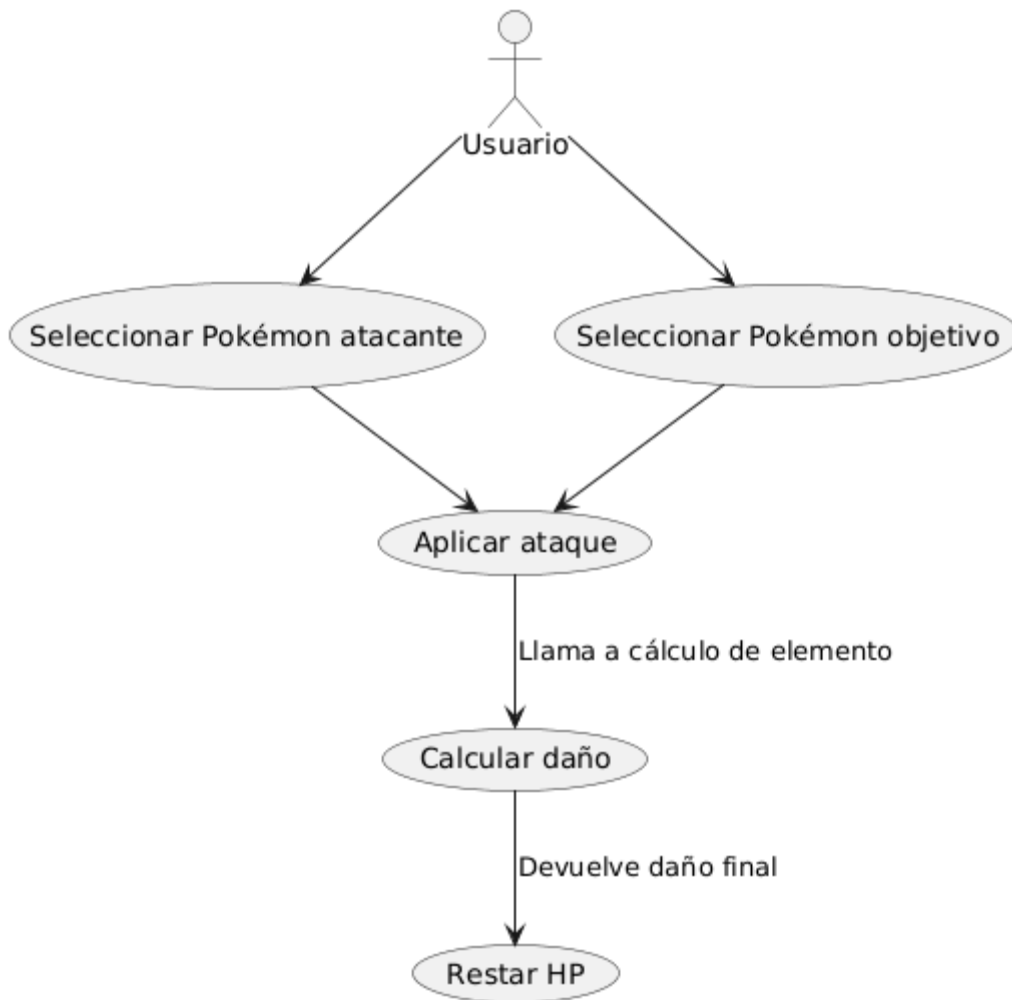
Este diagrama representa el flujo interno del ataque:
El Pokémon usa un ataque, el ataque consulta al elemento para determinar efectividad y daño aplicado.

Diagrama UML 6. Diagrama caso de uso “Batalla de Pokemones”



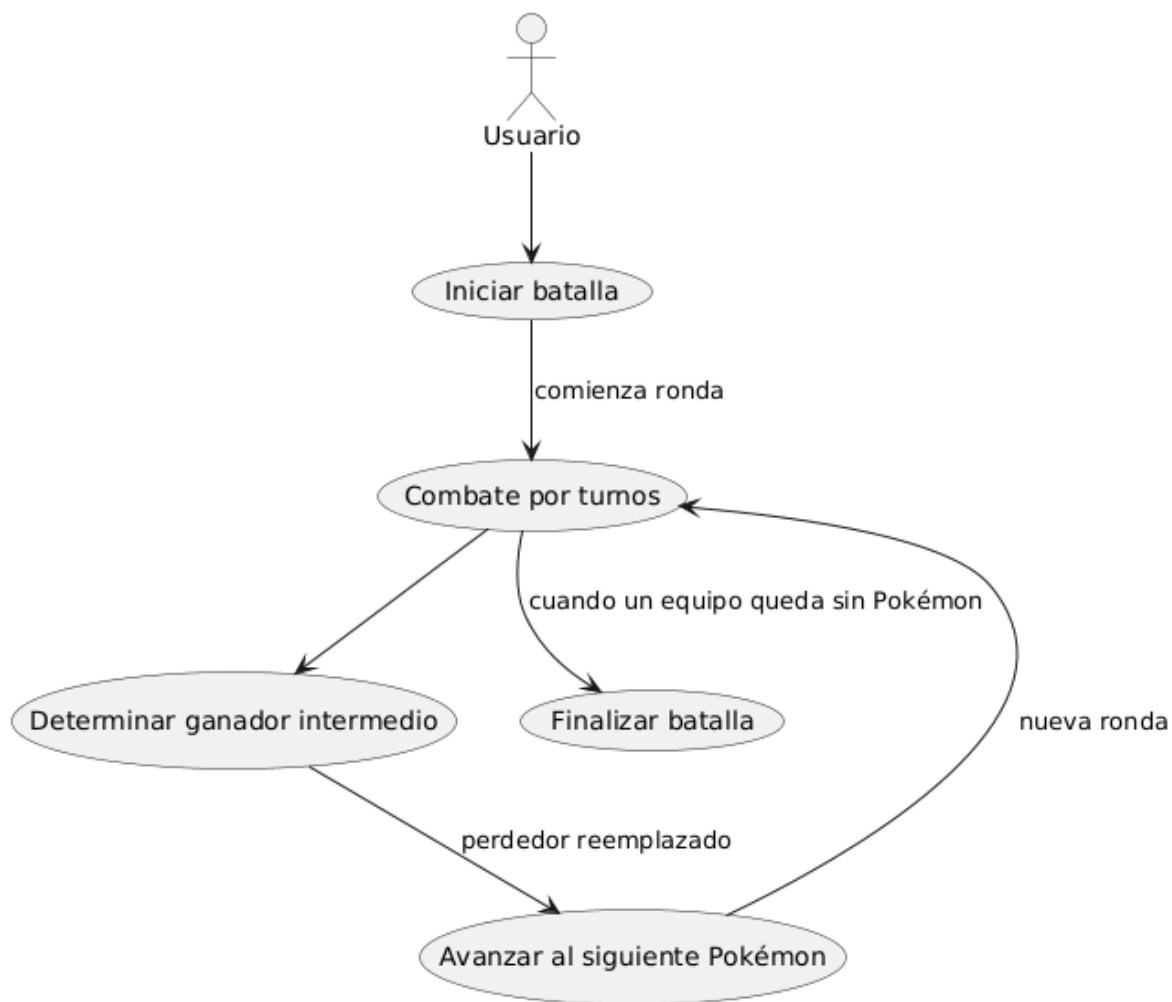
En este diagrama de caso de uso, se puede ver como se inicializara la batalla de pokemones en equipo, desde que Pokémon iniciara atacando, verificara el elemento del Pokémon al que atacara y que ataque lanzara, la vida que se le restara al Pokémon 2, podemos ver como el Pokémon 2 atacara a el Pokémon 1 y la vida que se le restara al Pokémon 1.

Diagrama UML 7. Caso de Uso: Proceso de Ataque



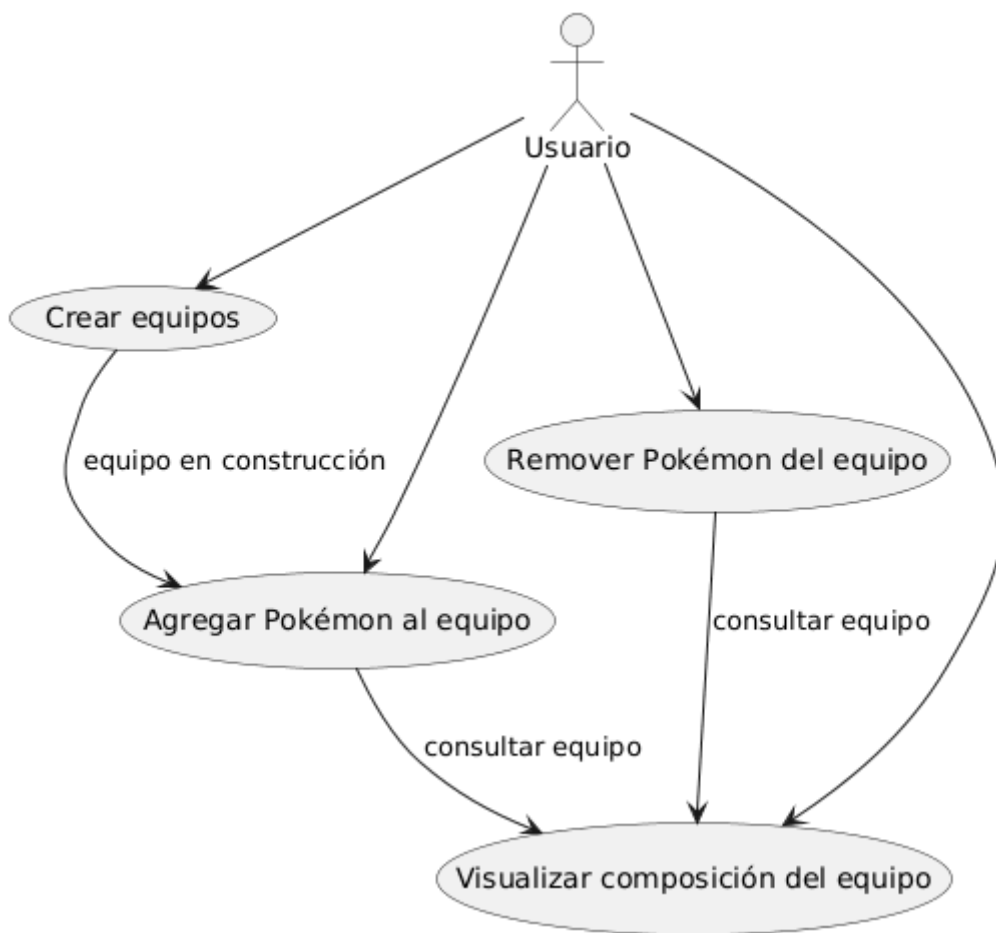
Este diagrama modela el proceso cuando un Pokémon ataca a otro. Desde la selección del atacante y objetivo hasta el cálculo de daño y afectación de HP.

Diagrama UML 8. Caso de Uso: Ciclo de Batalla



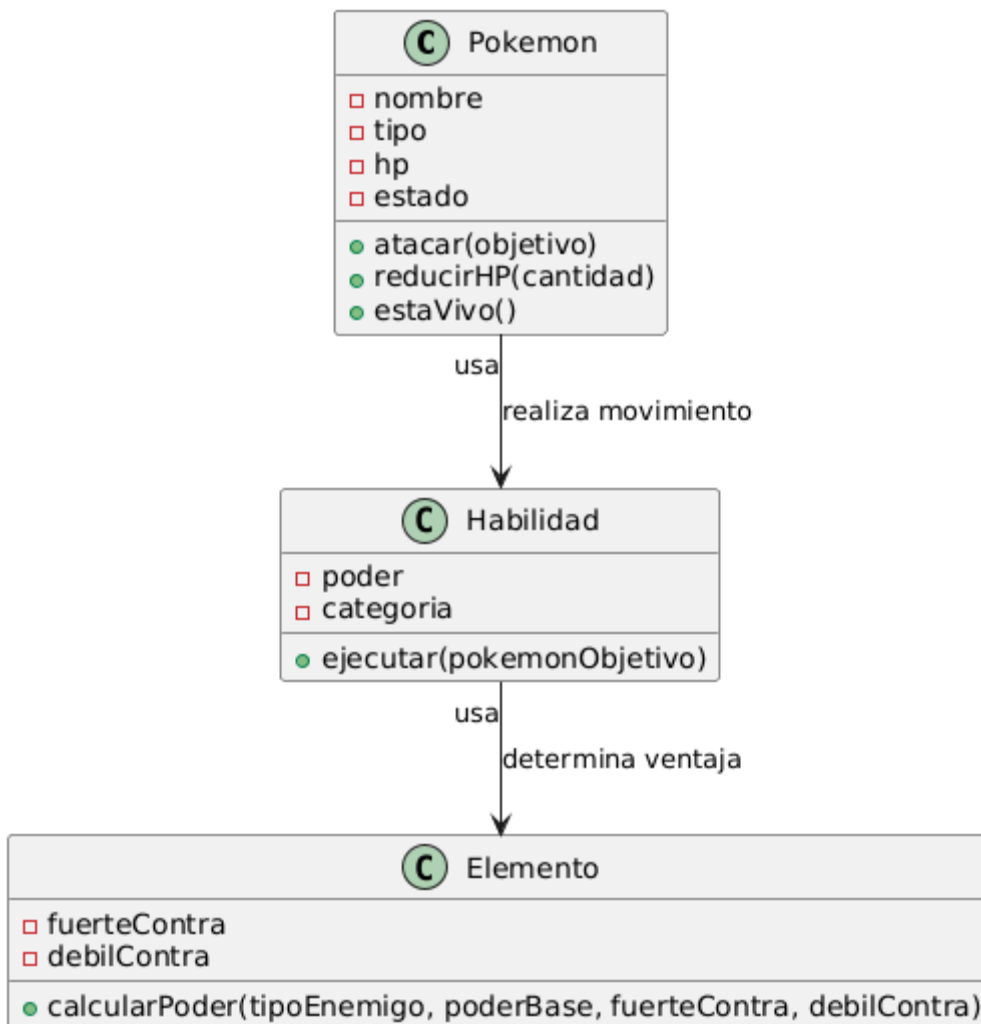
Este diagrama muestra el flujo de turnos y eliminación progresiva de Pokémon, hasta que uno de los equipos se queda sin participantes.

Diagrama UML 9. Caso de Uso: Gestión de Equipos



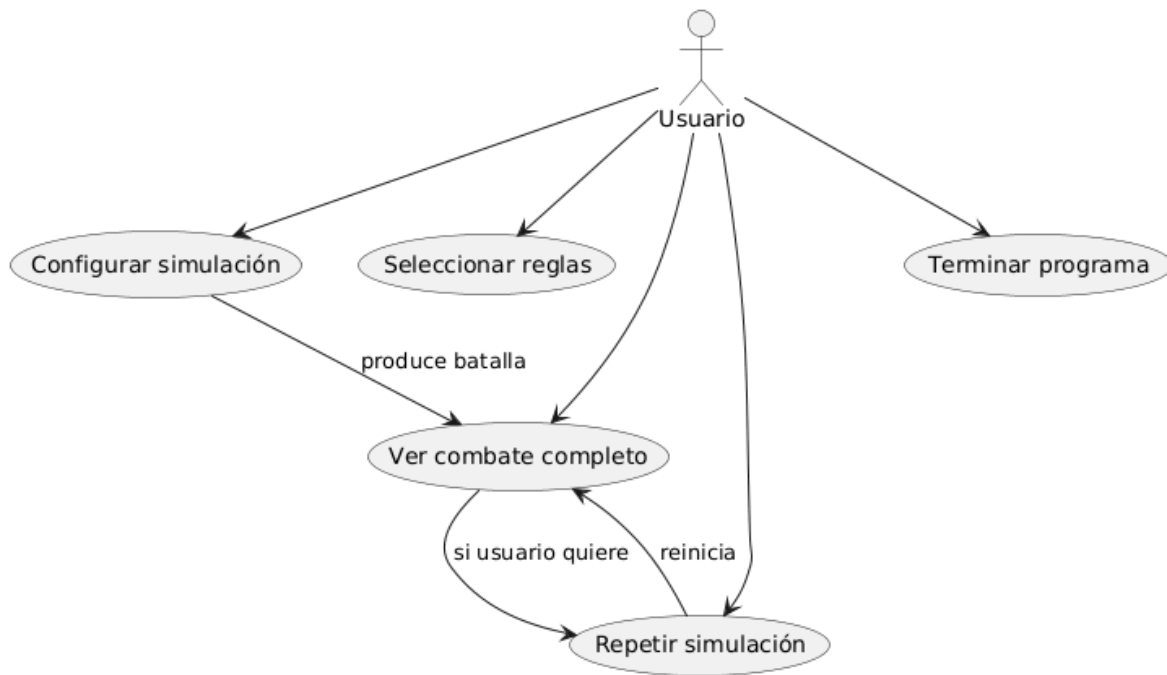
Este diagrama representa la preparación previa a la batalla y la administración de equipos por el usuario.

Diagrama UML 10. Diagrama de clases: Pokémon vs Habilidades



Este diagrama resume la relación entre Pokémon y sus habilidades, reforzando el concepto de composición y polimorfismo.

Diagrama UML 11. Diagrama de Caso de Uso: “Simulador completo”



Representa el uso global del simulador desde el punto de vista del usuario, mostrando interacción general con el sistema.

DOCUMENTACIÓN DEL CÓDIGO

Detalles del código

- En el diseño de este proyecto aplicamos Programación Orientada a Objetos, donde:
- Implementamos las clases: Batalla, Pokemon, Elemento y 3 ataques
- Definimos los atributos para pokemon: nombre, tipo, habilidades, salud y estado.
- Creamos el método de pokemon: atacar()
- Creamos los arrays de objetos pokemon: Equipo 1 y Equipo 2
- En la clase Batalla implementamos el método : luchar ()
- La clase Pokemon encapsula atributos y comportamientos de un Pokémon.
- La clase Elemento actúa como clase padre abstracta que provee un método para calcular la potencia efectiva de un ataque.
- Las clases Agua, Fuego y Planta heredan de Elemento utilizando herencia.
- Los ataques concretos como Hidrobomba, Cascada, Lanzallamas, etc. heredan de la clase de elemento correspondiente.
- La clase Batalla coordina la interacción entre los equipos, aplicando composición.

A continuación, el código en JavaScript, realizado en Visual Studio Code (lo pegue y le baje el tamaño de la fuente para no hacer tan largo el documento).

Código de JavaScript:

```
class Pokemon {  
  constructor(name, type, skills, hp, status){  
    this.name = name  
    this.type = type  
    this.skills = skills  
    this.hp = hp  
    this.status = status  
  }  
  
  attack(pokemon){  
    if(this.hp > 0){  
      console.log(this.name, "atacando a: ", pokemon.name);  
      let skill = this.skills[0]  
      let power = skill.verifyElement(pokemon.type, skill.power)  
      pokemon.reduceHP(power)  
    } else {  
      console.log(this.name, "no puede atacar, vida: ", this.hp);  
    }  
  }  
  
  isAlive(){  
    return this.hp > 0  
  }  
  
  reduceHP(power){  
    this.hp = this.hp - power  
    console.log(this.name, "ahora tiene hp:", this.hp)  
  }  
}
```

```

class Batalla{
  constructor(team1,team2){
    this.team1 = team1
    this.team2 = team2
  }

  luchar(){
    console.log("Batalla iniciada...");

    let indexTeam1=0
    let indexTeam2=0

    while(indexTeam1 < this.team1.length && indexTeam2 < this.team2.length){
      let pokemon1 = this.team1[indexTeam1]
      let pokemon2 = this.team2[indexTeam2]

      let survivor = this.atacarHastaDerrotar(pokemon1,pokemon2)
      if(survivor === pokemon1){
        indexTeam2++
      }else{
        indexTeam1++
      }
    }

    if(indexTeam1 >= this.team1.length){
      console.log("🔥 GANÓ EL EQUIPO 2 🔥");
    } else {
      console.log("🔥 GANÓ EL EQUIPO 1 🔥");
    }

    console.log("Equipo1:",this.team1);
  }
}

```

```

    console.log("Equipo2:",this.team2);
}

atacarHastaDerrotar(pokemon1,pokemon2){
    while(pokemon1.hp > 0 && pokemon2.hp > 0){
        pokemon1.attack(pokemon2)
        if(pokemon2.hp <= 0) break;
        pokemon2.attack(pokemon1)
    }

    if(pokemon1.hp > 0){
        console.log("Survivor: ",pokemon1.name);
        return pokemon1
    }else{
        console.log("Survivor: ",pokemon2.name);
        return pokemon2
    }
}
}

class Element{
    calculatePower(enemyElement,attack,strongAgainst,weakAgainst){
        if(enemyElement === weakAgainst){
            console.log("⚡ El ataque fue débil");
            return attack - 10
        } else if(enemyElement === strongAgainst){
            console.log("💥 El ataque fue FUERTE");
            return attack + 10
        }else{
            console.log("➡ El ataque fue normal");
            return attack
        }
    }
}

```

```
    }  
  }  
}
```

```
class Agua extends Element{  
  strongAgainst = "Fuego"  
  weakAgainst = "Planta"  
  verifyElement(enemyElement,attack){  
    return super.calculatePower(  
      enemyElement,  
      attack,  
      this.strongAgainst,  
      this.weakAgainst  
    )  
  }  
}
```

```
class Fuego extends Element{  
  strongAgainst = "Planta"  
  weakAgainst = "Agua"  
  verifyElement(enemyElement,attack){  
    return super.calculatePower(  
      enemyElement,  
      attack,  
      this.strongAgainst,  
      this.weakAgainst  
    )  
  }  
}
```

```
class Planta extends Element{
```

```
    strongAgainst = "Agua"
    weakAgainst = "Fuego"
    verifyElement(enemyElement,attack){
        return super.calculatePower(
            enemyElement,
            attack,
            this.strongAgainst,
            this.weakAgainst
        )
    }
}
```

```
class Cascada extends Agua{
    power = 102
}
class Hidrobomba extends Agua{
    power = 98
}
```

```
class Lanzallamas extends Fuego{
    power = 105
}
```

```
class HojaAfilada extends Planta{
    power = 100
}
```

```
const charmander1 = new Pokemon(
    "Charmander1",
    "Fuego",
    [new Lanzallamas()],
```

```
    1000,  
    true  
)  
const charmander22 = new Pokemon(  
    "Charmander2",  
    "Fuego",  
    [new Lanzallamas()],  
    1000,  
    true  
)
```

```
const squirtle1 = new Pokemon(  
    "Squirtle1",  
    "Agua",  
    [new Hidrobomba()],  
    900,  
    true  
)
```

```
const squirtle2 = new Pokemon(  
    "Squirtle2",  
    "Agua",  
    [new Cascada()],  
    900,  
    true  
)
```

```
const bulbasaur1 = new Pokemon(  
    "Bulbasaur1",  
    "Planta",  
    [new HojaAfilada()],  
    1100,
```

```
    true  
  )
```

```
const bulbasaur2 = new Pokemon(  
  "Bulbasaur2",  
  "Planta",  
  [new HojaAfilada()],  
  1100,  
  true  
)
```

```
const team1 = [charmander1,squirtle1,bulbasaur1]  
const team2 = [squirtle2,bulbasaur2,charmander22]
```

```
const battle = new Batalla(team1,team2)  
battle.luchar()
```

Explicación del Código

Clase: Pokemon

```
JS pokemon.js X
JS pokemon.js > Pokemon
1  class Pokemon {
2      constructor(name, type, skills, hp, status){
3          this.name = name
4          this.type = type
5          this.skills = skills
6          this.hp = hp
7          this.status = status
8      }
```

- Esta clase define la estructura general de un Pokémon.
- Estos this.name, this.type, etc., son atributos.
- Representan características de un Pokémon (vida, ataques, tipo, etc.).
- Aquí se demuestra encapsulamiento, ya que todo lo relacionado con un Pokémon está contenido dentro de la instancia del objeto.

Método: attack()

```
attack(pokemon){
    if(this.hp > 0){
        console.log(this.name, "atacando a: ", pokemon.name);
        let skill = this.skills[0]
        let power = skill.verifyElement(pokemon.type, skill.power)
        pokemon.reduceHP(power)
    } else {
        console.log(this.name, "no puede atacar, vida: ", this.hp);
    }
}
```

- Este método permite que un Pokémon ataque a otro Pokémon.
- this.skills[0] significa que el Pokémon solo usa su primer ataque.
- Aquí ocurre interacción entre objetos, ya que un Pokémon ejecuta métodos del otro (pokemon.reduceHP()).

- Si el Pokémon tiene HP menor o igual a cero, no puede atacar.

Método: isAlive()

```
isAlive(){  
    return this.hp > 0  
}
```

- Indica si un Pokémon sigue vivo.
- Retorna true si tiene vida, false si ya está derrotado.
- Es un método auxiliar de estado.

Método: reduceHP()

```
reduceHP(power){  
    this.hp = this.hp - power  
    console.log(this.name, "ahora tiene hp:", this.hp)  
}
```

- Reduce los puntos de vida del Pokémon de acuerdo con el poder del ataque recibido.
- Representa daño en combate.
- Actualiza el atributo interno hp.

Clase: Batalla

```
class Batalla{  
    constructor(team1,team2){  
        this.team1 = team1  
        this.team2 = team2  
    }  
}
```

- Representa la batalla entre dos equipos.
- Aplica composición, porque contiene listas de objetos Pokémon.

- Los Pokémon NO existen solos, sino dentro de un equipo dentro de una batalla.

Método: luchar()

```
luchar(){  
    console.log("Batalla iniciada...");  
  
    let indexTeam1=0  
    let indexTeam2=0
```

- Comienza la batalla.
- indexTeam1 e indexTeam2 representan cuál Pokémon está activo en cada equipo.

While:

```
while(indexTeam1 < this.team1.length && indexTeam2 < this.team2.length){
```

- Se ejecuta siempre que ambos equipos tengan Pokémon vivos disponibles.
- Es el ciclo de batalla.

Dentro del While:

```
let pokemon1 = this.team1[indexTeam1]  
let pokemon2 = this.team2[indexTeam2]
```

Selecciona el Pokémon actual de cada equipo.

```
let survivor = this.atacarHastaDerrotar(pokemon1,pokemon2)
```

Llama al método que hace que los dos Pokémon peleen hasta que uno sea derrotado.

```

if(survivor === pokemon1){
    indexTeam2++
}else{
    indexTeam1++
}

```

- El perdedor es reemplazado por el siguiente Pokémon del equipo.
- Esto representa avance de turno en un estilo torneo.

```

if(indexTeam1 >= this.team1.length){
    console.log("🔥 GANÓ EL EQUIPO 2 🔥");
} else {
    console.log("🔥 GANÓ EL EQUIPO 1 🔥");
}

```

- Determina cuál equipo se quedó sin Pokémon.
- Ese equipo es el perdedor.

Método: atacarHastaDerrotar()

```

atacarHastaDerrotar(pokemon1,pokemon2){
    while(pokemon1.hp > 0 && pokemon2.hp > 0){
        pokemon1.attack(pokemon2)
        if(pokemon2.hp <= 0) break;
        pokemon2.attack(pokemon1)
    }
}

```

- Aquí se define el combate 1 vs 1.
- Mientras ambos tengan vida, atacan por turnos.
- Es una simulación de enfrentamiento directo.

```

if(pokemon1.hp > 0){
    console.log("Survivor: ",pokemon1.name);
    return pokemon1
}else{
    console.log("Survivor: ",pokemon2.name);
    return pokemon2
}

```

- Retorna el Pokémon ganador del duelo individual.
- Sirve para procesar la lógica del torneo.

Clase: Element

```

class Element{
    calculatePower(enemyElement,attack,strongAgainst,weakAgainst){

```

- Esta clase define la lógica elemental general.
- No se usa directamente en batalla.
- Es clase PADRE de Agua, Fuego y Planta.
- Contiene el método común que se hereda.

```

if(enemyElement === weakAgainst){
    console.log("zz El ataque fue débil");
    return attack - 10

```

Si el ataque es débil contra el tipo enemigo:

1. Muestra el mensaje que el ataque fue debil
2. Baja el daño -10

```

} else if(enemyElement === strongAgainst){
    console.log("★ El ataque fue FUERTE");
    return attack + 10

```

Si el ataque es fuerte contra el tipo enemigo:

1. Muestra el mensaje que el ataque fue fuerte
2. Aumenta el daño +10

```

    }else{
        console.log("👉 El ataque fue normal");
        return attack
    }
}

```

Si el tipo elemental no da ventaja ni desventaja, daño normal.

Clases Derivadas de Element: Agua, Fuego, Planta y sus Ataques

Clase Agua:

```

class Agua extends Element{
    strongAgainst = "Fuego"
    weakAgainst = "Planta"
    verifyElement(enemyElement,attack){
        return super.calculatePower(
            enemyElement,
            attack,
            this.strongAgainst,
            this.weakAgainst
        )
    }
}

```

- Agua hereda de Element
- Tiene ventaja contra fuego
- Tiene desventaja contra planta

```

✓ class Cascada extends Agua{
    power = 102
}
✓ class Hidrobomba extends Agua{
    power = 98
}

```

Instanciación de ataques de agua

La clase Agua modela el elemento agua. Hereda comportamiento de la clase Element y define ventaja contra fuego y debilidad ante planta. Además sirve como base para ataques derivados como Hidrobomba y Cascada.

Clase Fuego:

```
class Fuego extends Element{
    strongAgainst = "Planta"
    weakAgainst = "Agua"
    verifyElement(enemyElement, attack){
        return super.calculatePower(
            enemyElement,
            attack,
            this.strongAgainst,
            this.weakAgainst
        )
    }
}
```

- Fuego hereda de Element
- Guarda sus ventajas y desventajas elementales
- Usa el método heredado calculatePower() del padre

```
class Lanzallamas extends Fuego{
    power = 105
}
```

Instanciación de ataques de fuego

La clase Fuego representa el elemento fuego dentro del sistema de combate. Hereda de Element y define ventajas contra Planta y desventajas contra Agua. Esta clase permite ajustar dinámicamente el daño según el tipo del oponente y se utiliza como padre de ataques derivados como Lanzallamas.

Clase Planta:

```
class Planta extends Element{
    strongAgainst = "Agua"
    weakAgainst = "Fuego"
    verifyElement(enemyElement, attack){
        return super.calculatePower(
            enemyElement,
            attack,
            this.strongAgainst,
            this.weakAgainst
        )
    }
}
```

- Planta hereda de Element
- Ventaja contra agua
- Debilidad contra fuego

```
class HojaAfilada extends Planta{
    power = 100
}
```

Instanciación de ataques de planta

La clase Planta representa el elemento vegetal. Gracias a la herencia desde Element, reutiliza la función de cálculo de daño elemental, definiendo ventaja contra agua y desventaja contra fuego. Esta clase es base para ataques como HojaAfilada.

Instanciación de Pokémon

En esa sección del programa, se están creando instancias reales de Pokémon a partir de la clase Pokemon.

Cada línea:

`const charmander1 = new Pokemon(...)`

```
const charmander22 = new Pokemon(  
  "Charmander22",  
  "Fuego",  
  [new Lanzallamas()],  
  1000,  
  true  
)
```

significa:

Crear un objeto en memoria que representa un Pokémon específico, con sus propias características y atributos.

En la construcción de cada Pokémon ocurre lo siguiente:

```
"Charmander1",  
"Fuego",  
[new Lanzallamas()],  
1000,  
true
```

Cuando el programa corre:

```
const charmander1 = new Pokemon(  
  "Charmander1",  
  "Fuego",  
  [new Lanzallamas()],  
  1000,  
  true  
)
```

Se reservan recursos en memoria para un nuevo objeto

Se crea un Pokémon con:

- name = "Squirtle1"
- type = "Agua"
- skills = [objeto Hidrobomba]
- hp = 900
- status = true

□ Ese objeto se almacena en la variable squirtle1

```
const charmander1 = new Pokemon(  
  "Charmander1",  
  "Fuego",  
  [new Lanzallamas()],  
  1000,  
  true  
)  
  
const charmander22 = new Pokemon(  
  "Charmander22",  
  "Fuego",  
  [new Lanzallamas()],  
  1000,  
  true  
)  
  
const squirtle1 = new Pokemon(  
  "Squirtle1",  
  "Agua",  
  [new Hidrobomba()],  
  900,  
  true  
)  
  
const squirtle2 = new Pokemon(  
  "Squirtle2",  
  "Agua",  
  [new Cascada()],  
  900,  
  true  
)  
  
const bulbasaur1 = new Pokemon(  
  "Bulbasaur1",  
  "Planta",  
  [new HojaAfilada()],  
  1100,  
  true  
)
```

Se interpreta así:

Parámetro	Significado
Name	Nombre individual del Pokémon
type	Elemento al que pertenece (agua, fuego, planta)
skills	Lista de ataques que puede usar (array)
hp	Puntos de vida iniciales
status	Estado vivo/activo (true)

Aquí se ve composición:

Un Pokémon contiene un ataque

Ejemplo:

[charmander1] → contiene [Lanzallamas()]

[squirtle2] → contiene [Cascada()]

[bulbasaur1] → contiene [HojaAfilada()]

Esto es **un objeto dentro de otro objeto**

En este bloque del código se instancian los Pokémon que participarán en la batalla. Cada objeto creado representa un Pokémon único en memoria y se construye utilizando el constructor de la clase Pokemon. A cada uno se le asignan sus atributos principales: nombre, tipo elemental, lista de ataques, puntos de vida iniciales y estado de actividad. Este proceso implementa el concepto de composición, ya que cada Pokémon contiene en su interior uno o más ataques como objetos.

Esto es algo conceptual:

- ✓ se crean objetos reales
- ✓ cada Pokémon existe independientemente
- ✓ cada uno tiene su propio HP
- ✓ cada uno tiene su propio tipo
- ✓ cada uno tiene su propio ataque
- ✓ se almacenan en variables para ser usados en batalla

Creación de equipos y Batalla

```
const team1 = [charmander1,squirtle1,bulbasaur1]
const team2 = [squirtle2,bulbasaur2,charmander22]

const battle = new Batalla(team1,team2)
battle.luchar()
```

- Se definen los equipos como arreglos.
- Se crea una instancia de Batalla.
- Se inicia la simulación.

Ejecución del programa:

Posición en la carpeta correcta del proyecto

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS

PS C:\Users\jalex\Desktop\P00_Pokemon> 
```

Ejecutamos el programa con Node.js:

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS

PS C:\Users\jalex\Desktop\P00_Pokemon> node pokemon.js
```

Parte del combate — por ejemplo:

```
Batalla iniciada...
Charmander1 atacando a: Squirtle2
zz El ataque fue débil
Squirtle2 ahora tiene hp: 805
Squirtle2 atacando a: Charmander1
* El ataque fue FUERTE
Charmander1 ahora tiene hp: 888
Charmander1 atacando a: Squirtle2
zz El ataque fue débil
Squirtle2 ahora tiene hp: 710
Squirtle2 atacando a: Charmander1
* El ataque fue FUERTE
```

Acciones en tiempo real durante la batalla entre Pokémon

Final del combate:

```
Charmander2 ahora tiene hp: 460
Charmander2 atacando a: Bulbasaur1
🔥 El ataque fue FUERTE
Bulbasaur1 ahora tiene hp: 110
Bulbasaur1 atacando a: Charmander2
⚡ El ataque fue débil
Charmander2 ahora tiene hp: 370
Charmander2 atacando a: Bulbasaur1
🔥 El ataque fue FUERTE
Bulbasaur1 ahora tiene hp: -5
Survivor: Charmander2
🔥 GANÓ EL EQUIPO 2 🔥
```

Resultado final

Durante la ejecución del programa, el simulador inicia una batalla por turnos entre los Pokémon definidos previamente. En cada turno, ambos Pokémon atacan uno al otro utilizando sus ataques específicos, modificando los puntos de vida según la efectividad del elemento contra el rival. El ciclo continúa hasta que uno de los Pokémon llega a cero puntos de vida, determinando al sobreviviente del duelo. El programa repite este proceso hasta que un equipo se queda sin Pokémon disponibles, indicando al final cuál equipo fue el ganador y mostrando los valores finales de los atributos de cada Pokémon.

Conclusión

Como equipo, consideramos que este proyecto fue una experiencia de aprendizaje muy valiosa, ya que aplicamos de forma práctica los conceptos de Programación Orientada a Objetos en un caso real: la simulación de un combate Pokémon por turnos.

A lo largo del desarrollo fuimos comprendiendo cómo las clases y objetos interactúan entre sí y cómo la herencia nos permitió reutilizar y extender comportamientos en las clases hijas, evitando duplicación de código. Además, el uso de composición nos ayudó a entender cómo un objeto puede contener a otros para formar estructuras complejas, como lo hicimos con los equipos de Pokémon.

También reforzamos la importancia del encapsulamiento del estado interno de los objetos y de la validación de comportamientos mediante métodos específicos. El proceso de prueba y ejecución nos permitió observar y analizar los resultados del combate en tiempo real, comprendiendo la lógica de turnos, ataques y efectos elementales.

Finalmente, reconocemos que este proyecto no solo fue útil para entender los fundamentos de la POO, sino también para trabajar de forma organizada, colaborativa y con responsabilidad compartida. Concluirlo exitosamente nos dejó no solo el programa funcional, sino también una comprensión más profunda del diseño orientado a objetos y su aplicación en la resolución de problemas

Agradecimiento

Queremos agradecer al creador del contenido del canal cuyo video tutorial utilizamos como referencia para desarrollar este proyecto:

<https://www.youtube.com/watch?v=dYuacYTH1EM>

Su explicación clara y práctica fue un punto de partida importante para comprender la lógica de combate por turnos y la estructura de clases relacionadas con los Pokémon y sus ataques. A partir de ese material, pudimos adaptar, modificar y extender el código para ajustarlo a los requerimientos de nuestra materia, así como para fortalecer nuestra comprensión de los conceptos de Programación Orientada a Objetos.

Agradecemos el aporte educativo de dicho autor, ya que contribuyó significativamente a nuestro aprendizaje y a la realización exitosa de este proyecto académico.